Tema 4: Interfaces Gráficas con Swing Dialog

Pablo Gordillo (pabgordi@ucm.es)

Departamento de Sistemas Informáticos y Computación

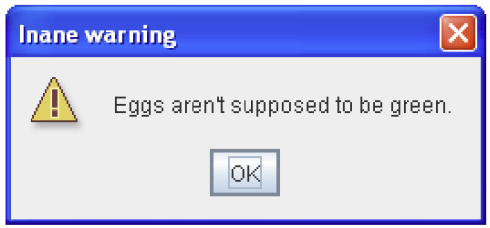
Facultad de Informática



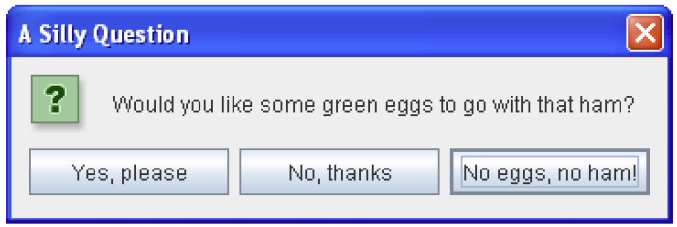
(Basado en el material creado por Puri Arenas, Jesús Correas y Samir Genaim)

Dialogs

- Una caja de diálogo es una subventana (que depende de un frame) que sirve para mostrar información al usuario (posibles errores, warnings, etc.).
- Solo vamos a ver los modales, es decir, los que paran la aplicación hasta que desaparece el cuadro de diálogo.
- Existen distintos tipos de cajas de diálogo. Las más utilizadas son:



JOptionPane.showMessageDialog(frame,...)



JOptionPane.showOptionDialog(frame,...)

ShowMessageDialogs

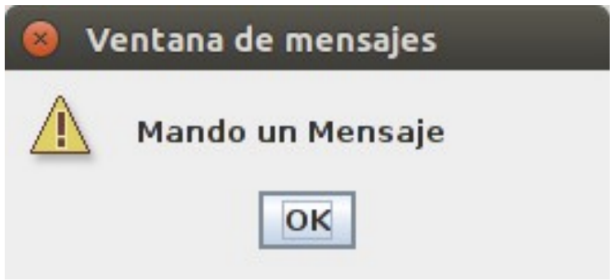
- Lo más habitual es lanzar la ventana con:
  JOptionPane.showMessageDialog(frame,String1);
  JOptionPane.showMessageDialog(frame,String1,String2,
  JOptionPane.icono);
- **frame** puede ser **null** o alguna ventana que ya exista (por ejemplo, la ventana principal). El diálogo pertenece a la ventana que se indique, lo que significa que cuando la ventana se maximiza/minimiza/mueve/destruye etc, también lo hace el diálogo. Sin embargo si ponemos frame a **null**, entonces el diálogo es independiente.
- **String1** es el mensaje que se muestra en la ventana. **String2** es el título de la ventana (por defecto es Message). Icono es el icono que deseamos que se muestre (warning, error, etc.)

ShowMessageDialog

- **JOptionPane.showMessageDialog(mainPanel, “Mando un Mensaje”);**



- **JOptionPane.showMessageDialog(mainPanel, “Mando un Mensaje”, “Ventana de mensajes”, JOptionPane.WARNING_MESSAGE);**

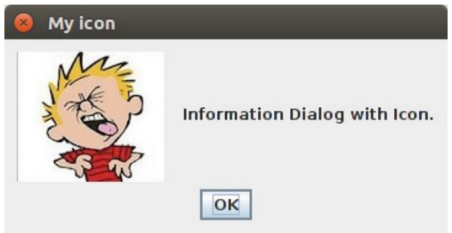


ShowMessageDialog










```
JOptionPane.showMessageDialog(frame,
    "Information Dialog (dialog by default).");

JOptionPane.showMessageDialog(frame,
    "Information Dialog (dialog by default).",
    "Message", JOptionPane.INFORMATION_MESSAGE);

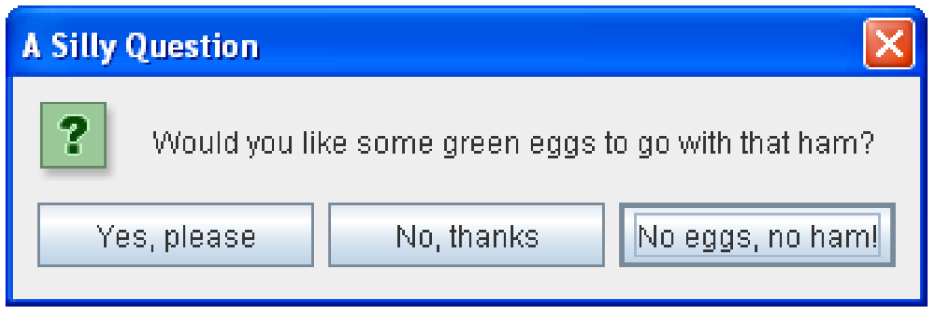
JOptionPane.showMessageDialog(frame,
    "Warning Dialog.", "Warning Icon",
    JOptionPane.WARNING_MESSAGE);


JOptionPane.showMessageDialog(frame,
    "Error Dialog.", "Error Icon",
    JOptionPane.ERROR_MESSAGE);

ImageIcon icon = createImageIcon("calvin.jpg");
JOptionPane.showMessageDialog(frame,
    "Information Dialog with Icon.",
    "My icon", JOptionPane.INFORMATION_MESSAGE,
    icon);
```

ShowOptionDialog

- Muestra un diálogo en el que pueden aparecer botonos especificados por el usuario, iconos, mensajes, etc.
- Ofrece una serie de botones a los que el usuario puede poner el texto, y su comportamiento se fija a partir de unas constantes.



ShowOptionDialog

- Para crear la caja de diálogo usamos:
  int n = JoptionPane.showOptionDialog(Parent, Message, Title, OptionType, MessageType, Icon, Options, InitialValue)
  donde:
  - **Parent**: Componente padre, que debe ser un **Frame** o **null**
  - **Message**: Texto que queremos que aparezca en el diálogo.
  - **Title**: Título de la ventana de diálogo.
  - **OptionType**: Conjunto de botones que aparecen en el diálogo: DEFALUT_OPTION, YES_NO_OPTION, YES_NO_CANCEL_OPTION,OK_CANCEL_OPTION
  - **MessageType**: Determina el tipo de icono que queremos en el diálogo: PLAIN_MESSAGE,ERROR_MESSAGE,INFORMATION_MESSAGE, QUESTION_MESSAGE
  - **Icon**: Si se quiere mostrar icono en la caja
  - **Object[ ] Options**: Texto que se desea que aparezca en cada botón.
  - **Object initialValue**: Botón que tendrá el foco.

ShowOptionDialog

int n = JOptionPane.showOptionDialog(...)

El parámetro **n** indica el botón seleccionado:

0: YES_OPTION,OK_OPTION

1: NO_OPTION

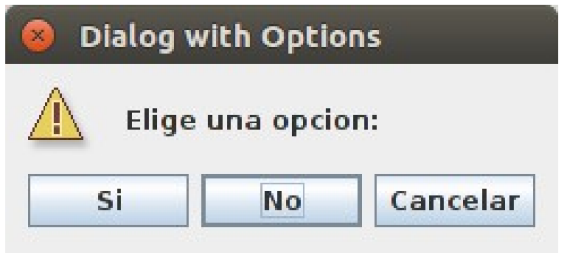
2: CANCEL_OPTION

-1: CLOSED_OPTION

ShowOptionDialog

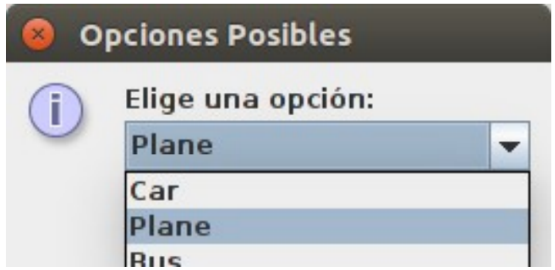
```
Object[] options = {"Si","No","Cancelar"};
int n = JOptionPane.showOptionDialog(mainPanel,
        "Elige una opci\'on:", "Opciones Posibles",
        JOptionPane.YES_NO_CANCEL_OPTION,
        JOptionPane.WARNING_MESSAGE,
        null ,
        options ,
        options[1]) ;

if (n==JOptionPane.YES_OPTION)
    System.out.println ("Has elegido SI");
else if (n==JOptionPane.NO_OPTION)
    System.out.println ("Has elegido NO");
else if (n==JOptionPane.CANCEL_OPTION)
    System.out.println ("Has elegido CANCELAR");
else
    System.out.println ("Has cerrado sin elegir ");
```



ShowInputDialog

Permite al usuario elegir una opción y la devuelve en un objeto de la clase Object, que habitualmente es un String.




```
Object[] possibilities = {"Car","Plane","Bus"};

String n = (String) JOptionPane.showInputDialog(
    this, // contenedor padre
    "Elige una opci\'on:", // mensaje en la ventana
    "Opciones Posibles", // etiqueta de la ventana
    JOptionPane.INFORMATION_MESSAGE, // icono seleccionado
    null, // icono seleccionado por el usuario (Icon)
    possibilities , // opciones para seleccionar
    "Plane");

if (n.equals("Car")) System.out.println ("Has elegido Car");
else if (n.equals("Plane")) System.out.println ("Has elegido Plane");
else if (n.equals("Bus")) System.out.println ("Has elegido Bus");
else System.out.println ("Has cerrado sin elegir ");
```

JFileChooser

- Permite mostrar una ventana para la selección de un fichero.
- Se crea con:

  JFileChooser fileChooser) new JFileCHooser();

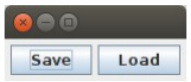
- Para mostrar la ventana:

  int slection = fileChooser.showXXXDialog(component);

  donde **component** puede ser cualquier componente que exista en alguna ventana de la aplicación. Nuestro **fileChooser** siempre aparecerá delante de dicha ventana.
  XXX puede ser **Open** o **Save**, dependiendo de si el fichero seleccionado lo queremos leer o escribir.
- Hasta que no se elija un fichero, el **fileChooser** estará abierto. Al elegirlo devolverá (**selection** tomará el valor):
  JFileChooser.CANCEL_OPTION: Si el usuario le ha dado al botón cancelar.
  JFileChooser.APPROVE_OPTION: Si el usuario le ha dado al botón aceptar.

JFileChooser

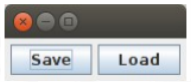
```
JButton save, load;
JFileChooser fc;
...
public void initGUI(){
  ...
  this.fc = new JFileChooser();
  ...
}
@Override
public void actionPerformed(ActionEvent e) {
  if (e.getSource() == this.load) {
    int v = fc.showOpenDialog(null);
    if (v==JFileChooser.APPROVE_OPTION){
      File file = fc.getSelectedFile(); //obtenemos el fichero seleccionado
      System.out.println("loading " +file.getName());
    }
    else System.out.println("load cancelled by user");
  }
```





JFileChooser

```
JButton save, load;
JFileChooser fc;
...
public void initGUI(){
  ...
  this.fc = new JFileChooser();
  ...
}
@Override
public void actionPerformed(ActionEvent e) {
  if (e.getSource() == this.load) {
    int v = fc.showOpenDialog(null);
    if (v==JFileChooser.APPROVE_OPTION){
      File file = fc.getSelectedFile();
      System.out.println("loading " +file.getName());
    }
    else System.out.println("load cancelled by user");
  }
  else if (e.getSource()==this.save){
    int v = fc.showSaveDialog(null);
    if (v==JFileChooser.APPROVE_OPTION){
      File file = fc.getSelectedFile();
      System.out.println("saving " + file.getName());
    }
    else System.out.println("load cancelled by user");
  }
}
```







JFileChooser

```
JButton save, load;
JFileChooser fc;
...
public void initGUI(){
  ...
  this.fc = new JFileChooser();
  ...
}
@Override
public void actionPerformed(ActionEvent e) {
  if (e.getSource() == this.load) {
    int v = fc.showOpenDialog(null);
    if (v==JFileChooser.APPROVE_OPTION){
      File file = fc.getSelectedFile();
      System.out.println("loading " +file.getName());
    }
    else System.out.println("load cancelled by user");
  }
  else if (e.getSource()==this.save){
    int v = fc.showSaveDialog(null);
    if (v==JFileChooser.APPROVE_OPTION){
      File file = fc.getSelectedFile();
      System.out.println("saving " + file.getName());
    }
    else System.out.println("load cancelled by user");
  }
}
```

Si el **fileChooser** se crea una sola vez, entonces se abre directamente en el directorio en el que se encontraba la última vez.





JFileChooser

- Un fileChooser se abre por defecto en el directorio HOME del usuario. Para elegir el directorio en el que se abre:

  fileChooser.setCurrentDirectory(Directorio)

- Por defecto solo permite elegir ficheros. Para elegir también directorios hay que usar:

  fileChooser.setFilesSelectionMode(Option)

  donde **Option** puede ser:
  JFileChooser.FILES_ONLY
  JFileChooser.DIRECTORIES_ONLY
  JFileChooser.FILES_AND_DIRECTORIES